Alan Rossini Paulino — RA: 25362408-2
Gabriel Damião de Castro — RA: 25203195-2
Guilherme Augusto Gouvea — RA: 25357973-2
Pedro Lucas Cabral dos Santos — RA: 25033058-2

ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

OPERAÇÃO SENTINELA LONDRINA:
Arquitetura de Software para Monitoramento de Arboviroses via Dados Abertos

Atividade de Estudo Programado — 1.º Bimestre — 2026.1

LONDRINA
2026

SUMÁRIO

1 INTRODUÇÃO

1.1 Objetivos

1.2 Proposta e Justificativa

2 FUNDAMENTAÇÃO TEÓRICA

3 DESENVOLVIMENTO DO SISTEMA

3.1 Ambiente

3.2 Recursos a Serem Utilizados

3.3 Diagramas

3.3.1 Diagrama de Caso de Uso

3.3.2 Especificação dos Casos de Uso

3.3.3 Diagrama de Classes

3.4 Protótipos

CONSIDERAÇÕES FINAIS

REFERÊNCIAS

1 INTRODUÇÃO

O monitoramento de arboviroses — dengue, zika e chikungunya — representa um desafio contínuo para os gestores de saúde pública no Brasil. A cidade de Londrina, no Paraná, enfrenta surtos cíclicos dessas doenças, especialmente durante os meses mais quentes, período em que a proliferação do Aedes aegypti se intensifica. A Secretaria Municipal de Saúde (SMS) de Londrina publica periodicamente boletins epidemiológicos que reúnem dados de notificações, casos confirmados, descartados e óbitos, discriminados por região da cidade.

Apesar da disponibilidade dessas informações, elas se encontram em formatos não estruturados — textos em PDF e planilhas dispersas —, o que dificulta a análise automatizada, a identificação de tendências temporais e a tomada de decisão ágil por parte dos servidores da saúde. Essa lacuna entre a geração do dado bruto e sua transformação em inteligência epidemiológica constitui o problema central que o presente trabalho se propõe a endereçar.

Nesse contexto, foi concebido o sistema Sentinela Londrina: uma solução computacional modular, desenvolvida em C++ e Python, capaz de minerar dados dos boletins epidemiológicos públicos, organizá-los em estruturas de dados dinâmicas — especificamente listas encadeadas implementadas manualmente — e gerar relatórios situacionais que apoiem a visualização da evolução das arboviroses por região.

1.1 Objetivos

Desenvolver um sistema computacional modular, denominado Sentinela Londrina, capaz de coletar, organizar e analisar dados epidemiológicos referentes às arboviroses em Londrina, utilizando estruturas de dados dinâmicas e princípios de programação orientada a objetos, de forma a transformar registros brutos em informações situacionais acionáveis.

São objetivos específicos do projeto:

- Modelar o domínio epidemiológico por meio de classes orientadas a objetos, contemplando herança, encapsulamento e polimorfismo.
- Implementar manualmente uma lista encadeada dinâmica em C++, com operações de inserção ordenada, busca e remoção.
- Desenvolver um módulo de ingestão (parser) em Python com suporte à API Gemini, capaz de extrair dados de boletins em PDF.

- Calcular indicadores epidemiológicos — taxa de positividade, taxa de crescimento e índice de risco — e exportá-los em JSON.
- Desenvolver uma interface de visualização em Next.js que exiba os dados em dashboard interativo com filtros por localidade.

1.2 Proposta e Justificativa

A Secretaria Municipal de Saúde de Londrina disponibiliza, mensalmente, boletins epidemiológicos relativos às arboviroses transmitidas pelo Aedes aegypti. Esses documentos contêm dados estratégicos como: número de casos notificados e confirmados, casos de dengue grave, casos de zika e chikungunya, além do número de óbitos, segmentados por regiões da cidade (Norte, Sul, Leste, Oeste, Central e Distritos Rurais).

Contudo, esses dados encontram-se publicados em formato PDF, com estrutura variável entre boletins, o que impede sua utilização direta por sistemas computacionais. A ausência de uma arquitetura de software adequada para consumir, estruturar e analisar essas informações resulta em três obstáculos técnicos concretos: a inconsistência na estrutura dos dados; a dificuldade de recuperação cronológica; e a abstração deficiente do domínio epidemiológico.

O sistema Sentinela Londrina propõe-se a superar esses obstáculos por meio de um pipeline computacional que integra mineração de dados com inteligência artificial (API Gemini), estruturas de dados dinâmicas implementadas em C++ e uma interface de visualização moderna em Next.js. O projeto justifica-se pela necessidade de transformar dados epidemiológicos dispersos em informação acionável para apoiar a tomada de decisão em saúde pública no município de Londrina.

2 FUNDAMENTAÇÃO TEÓRICA

A escolha de uma estrutura de dados adequada é determinante para a eficiência de qualquer sistema computacional. Ziviani (2011) aponta que estruturas dinâmicas, como as listas encadeadas, permitem que o sistema gerencie a memória de forma flexível, adaptando-se ao volume de dados que cresce conforme novas informações são coletadas.

No contexto do projeto Sentinela Londrina, a utilização de estruturas estáticas seria insuficiente para lidar com o crescimento imprevisível dos registros durante picos epidêmicos. A lista encadeada é uma estrutura de dados linear em que cada elemento, denominado nó, armazena um valor e um ponteiro para o próximo nó da sequência. Diferentemente dos arrays, os nós não precisam estar armazenados em posições contíguas de memória, conferindo flexibilidade na inserção e remoção de elementos sem a necessidade de realocação de todo o bloco de memória. Essa característica é descrita por Cormen et al. (2022) como fundamental das listas encadeadas ordenadas.

A Programação Orientada a Objetos (POO) é um paradigma de desenvolvimento que organiza o software em torno de objetos — instâncias de classes que encapsulam dados e comportamentos. Segundo Booch et al. (2007), os quatro pilares da POO são: encapsulamento, herança, polimorfismo e abstração. No sistema Sentinela Londrina, esses pilares são aplicados de forma sistemática: o encapsulamento é garantido pela declaração de atributos como privados; a herança modela a diferença entre regiões urbanas e distritos rurais; e o polimorfismo permite tratar objetos distintos de forma uniforme por meio de ponteiros do tipo Locality*.

A vigilância epidemiológica moderna depende da transformação contínua de dados brutos em informação estratégica. Tauil (2002) ressalta que o uso de tecnologias de informação para processar dados de infestações permite a identificação precoce de áreas de risco e a otimização dos recursos públicos no combate ao Aedes aegypti. A integração com modelos de linguagem de grande porte — como o Gemini, adotado no presente projeto para extração de dados de PDFs — representa um avanço significativo nessa direção, conforme destacado por Huang (2023).

3 DESENVOLVIMENTO DO SISTEMA

O sistema Sentinela Londrina é composto por três camadas principais, cada uma com responsabilidades bem delimitadas, que se comunicam por meio de contratos de dados definidos. O fluxo de dados completo pode ser resumido como: PDF (boletim epidemiológico) → Parser Python (Gemini API) → CSV → Backend C++ (LinkedList) → result.json → Dashboard Next.js.

3.1 Ambiente

O desenvolvimento do sistema foi realizado em ambiente Linux, utilizando as seguintes tecnologias e versões:

Camada	Tecnologia	Versão	Finalidade
Parser	Python	3.11+	Ingestão e extração de dados de PDFs
Backend	C++	C++17	Estruturas de dados e análise epidemiológica
Frontend	Next.js / React	14.x	Dashboard interativo de visualização
IA	Gemini API	2.5 Flash	Extração estruturada de tabelas em PDF
Dados	CSV / JSON	—	Formato intermediário e contrato de dados

3.2 Recursos a Serem Utilizados

Para o desenvolvimento e execução do sistema Sentinela Londrina, são necessários os seguintes recursos:

- Compilador GCC/G++ com suporte a C++17 para o módulo backend.
- Python 3.11+ com as bibliotecas: google-generativeai (API Gemini), PyPDF2 ou pdfplumber (leitura de PDFs) e pandas (manipulação de CSV).
- Node.js 18+ e npm para execução do frontend em Next.js.
- Chave de API do Google Gemini 2.5 Flash para o módulo de extração de dados.
- Boletins epidemiológicos em PDF da Secretaria Municipal de Saúde de Londrina (disponíveis em blog.londrina.pr.gov.br).
- Editor de código-fonte (VS Code recomendado) e sistema de controle de versão Git.

ID Descrição do Requisito Prioridade Status

RF-01

O sistema deve ler arquivos CSV ou TXT contendo dados minerados dos boletins epidemiológicos da Prefeitura de Londrina e popular a lista encadeada.

Alta Pendente

RF-02

O sistema deve organizar os registros em uma Lista Encadeada dinâmica, com cada nó representando uma Região (Norte, Sul, Leste, Oeste, Centro e Distritos).

Alta Pendente

RF-03

O sistema deve gerar um relatório situacional que aponte a região com maior índice de infestação (LIRAa).

Média Pendente

Legenda de Prioridade: ■ Alta ■ Média ■ Baixa

Os requisitos não funcionais definem restrições, qualidades e regras que o sistema deve respeitar,

independentemente das funcionalidades.

ID Descrição do Requisito Prioridade Status

RNF-01

O sistema deve suportar crescimento imprevisível de registros durante picos epidêmicos sem uso de estruturas estáticas de tamanho fixo.

Alta Pendente

Legenda de Prioridade: ■ Alta ■ Média ■ Baixa

3.3 Diagramas

3.3.1 Diagrama de Caso de Uso

O diagrama de caso de uso a seguir apresenta os dois atores do sistema — o Analista de Saúde, responsável pela consulta e visualização dos dados, e o Sistema Parser, responsável pela ingestão automática dos boletins — e seus respectivos casos de uso. A relação «include» entre os casos de uso Importar Boletim PDF e Processar com Gemini API indica que a importação de um boletim sempre aciona o processamento via inteligência artificial.

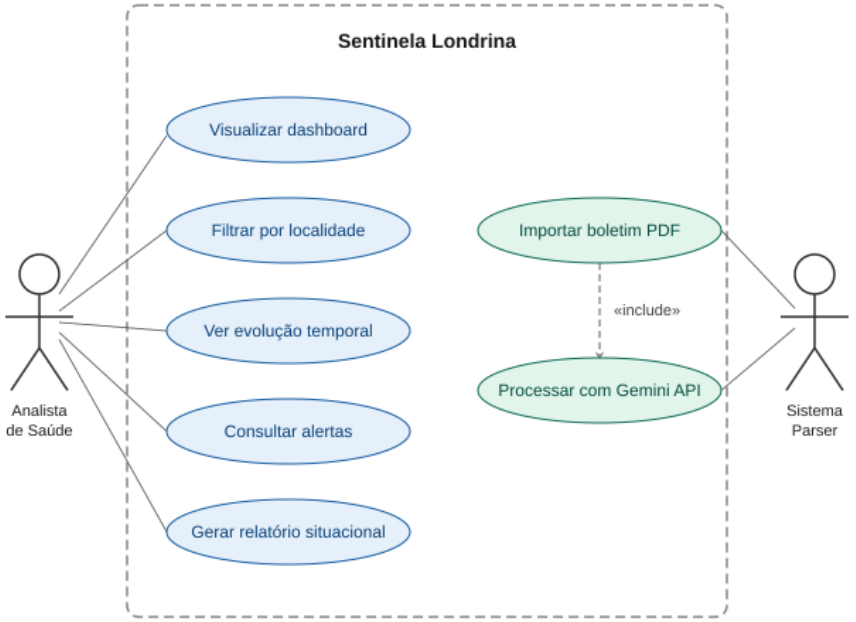

Figura 1 — Diagrama de Caso de Uso do sistema Sentinela Londrina

3.3.2 Especificação dos Casos de Uso

A seguir são apresentadas as especificações dos principais casos de uso do sistema Sentinela Londrina.

Campo	Descrição
Caso de Uso	UC01 — Visualizar Dashboard
Ator	Analista de Saúde
Pré-condição	O arquivo result.json deve estar disponível no servidor.
Fluxo Principal	1. O analista acessa o dashboard. 2. O sistema carrega o result.json. 3. São exibidos os indicadores consolidados: total de casos confirmados, óbitos e regiões em estado crítico.
Pós-condição	O analista visualiza o painel com os dados mais recentes.
Fluxo Alternativo	Se o result.json não for encontrado, o sistema exibe mensagem de erro orientando a executar o backend.

Campo	Descrição
Caso de Uso	UC02 — Filtrar por Localidade
Ator	Analista de Saúde
Pré-condição	O dashboard deve estar carregado com dados do result.json.
Fluxo Principal	1. O analista seleciona uma região no filtro. 2. O sistema filtra os dados da lista encadeada pela região selecionada. 3. São exibidos o histórico de boletins e os indicadores da região.
Pós-condição	O analista visualiza os dados específicos da localidade selecionada.
Fluxo Alternativo	Se a região não possuir boletins cadastrados, o sistema exibe mensagem informativa.

Campo	Descrição
Caso de Uso	UC03 — Importar Boletim PDF
Ator	Sistema Parser
Pré-condição	O arquivo PDF deve estar disponível no diretório data/raw/.
Fluxo Principal	1. O script download_bulletins.py baixa os PDFs do blog da Prefeitura. 2. O script ai_scan.py lê o arquivo PDF. 3. O conteúdo é enviado à API Gemini (UC04 incluído). 4. O CSV resultante é salvo em data/processed/.
Pós-condição	Um arquivo CSV com os dados do boletim está disponível para o backend.
Fluxo Alternativo	Se a API Gemini retornar erro, o script registra o log e pula o boletim.

Campo	Descrição
Caso de Uso	UC04 — Processar com Gemini API (include de UC03)
Ator	Sistema Parser
Pré-condição	O conteúdo binário do PDF deve estar disponível e a chave de API configurada.
Fluxo Principal	1. O conteúdo do PDF é codificado em base64. 2. É enviado à API Gemini 2.5 Flash com prompt estruturado. 3. A resposta JSON é convertida para CSV com as colunas do contrato de dados.
Pós-condição	Dados epidemiológicos estruturados disponíveis para o backend C++.
Fluxo Alternativo	Se a resposta não contiver os campos esperados, o parser utiliza valores zero como fallback.

3.3.3 Diagrama de Classes

O diagrama de classes UML a seguir representa a estrutura do sistema, composta por seis classes principais. A classe abstrata Locality define a interface comum para todas as regiões, enquanto UrbanRegion e RuralDistrict especializam o comportamento do método calculateRisk(). A LinkedList armazena ponteiros Locality* por meio de nós Node, permitindo polimorfismo em tempo de execução. Cada localidade mantém uma sublista encadeada de objetos MonthlyBulletin com o histórico de boletins mensais.

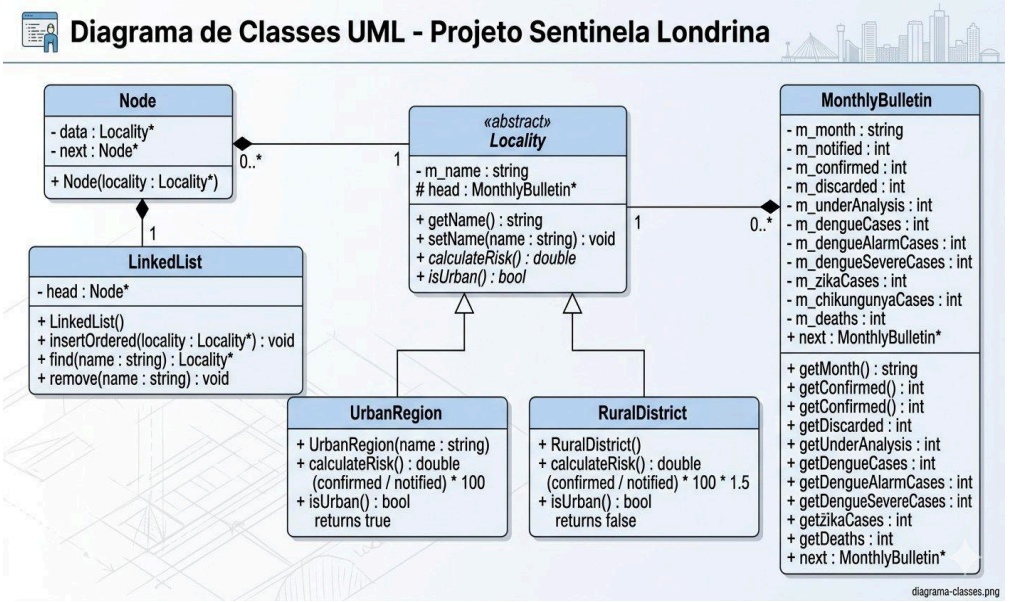

Figura 2 — Diagrama de Classes UML do sistema Sentinela Londrina

3.4 Protótipos

A prototipagem em baixo nível da interface de visualização foi desenvolvida com o objetivo de validar os fluxos de navegação e a organização da informação antes da implementação definitiva. Foram criados quatorze wireframes representando as principais telas do dashboard.

As telas prototipadas abrangem: a página de Visão Geral, com indicadores consolidados de casos confirmados, óbitos e regiões em estado crítico; a página de Evolução Temporal, com gráficos de linha mostrando a progressão mensal dos casos por região; a página de Relatório Situacional, que exibe o ranking das regiões por índice de risco; a página de Alertas, que destaca as regiões que atingiram o estado crítico; e a página de Filtros por Localidade, que permite ao usuário selecionar uma região específica e visualizar seu histórico completo em ordem cronológica.

A prototipagem foi realizada com fidelidade suficiente para guiar a implementação do frontend em Next.js, que resultou em nove páginas funcionais com dados reais provenientes do arquivo result.json gerado pelo backend C++. Os indicadores de status seguem a classificação: normal (taxa de positividade abaixo de 10%), alerta (entre 10% e 20%) e crítico (acima de 20%), alinhados às categorias do sistema de vigilância epidemiológica.

CONSIDERAÇÕES FINAIS

O desenvolvimento do sistema Sentinela Londrina demonstra, de forma concreta, a aplicação integrada dos conceitos de Engenharia de Software, Estruturas de Dados e Programação Orientada a Objetos no contexto de um problema real de saúde pública.

A implementação manual da lista encadeada em C++ evidencia a compreensão profunda das estruturas de dados dinâmicas: a gestão explícita de ponteiros, a alocação e liberação de memória com new e delete, e as operações de inserção ordenada e busca demonstram domínio técnico que vai além do uso de bibliotecas abstratas. A modelagem orientada a objetos, com a hierarquia Locality/UrbanRegion/RuralDistrict e o uso de polimorfismo para unificar o cálculo de risco, garante ao sistema flexibilidade e extensibilidade.

A integração com a API Gemini para extração de dados de boletins em PDF representa uma abordagem inovadora para o problema de mineração de dados governamentais não estruturados, resultando em dez boletins mensais reais processados e disponíveis para análise.

Para o 2.º bimestre, planeja-se: a refatoração do código com base no feedback dos professores; a implementação completa das funções de análise situacional, incluindo rankings de risco e alertas; o aprimoramento da interface gráfica do dashboard; e a elaboração do diagrama UML final atualizado. O projeto demonstra que a tecnologia computacional pode ser um instrumento poderoso na transformação de dados epidemiológicos dispersos em inteligência acionável para a saúde pública de Londrina.

REFERÊNCIAS

BOOCH, Grady; RUMBAUGH, James; JACOBSON, Ivar. UML: Guia do Usuário. 2. ed. Rio de Janeiro: Elsevier, 2007.

CORMEN, Thomas H. et al. Algoritmos: Teoria e Prática. 4. ed. Rio de Janeiro: LTC, 2022.

HUANG, Kalley. ChatGPT força universidades dos EUA a mudar. Estadão, 29 jan. 2023. Disponível em: https://www.estadao.com.br/link/cultura-digital/chatgpt-forca-universidades-dos-eua-a-mudar. Acesso em: 12 fev. 2026.

LONDRINA. Secretaria Municipal de Saúde. Boletins Epidemiológicos de Dengue, Zika e Chikungunya. Blog da Prefeitura de Londrina, 2025. Disponível em: https://blog.londrina.pr.gov.br/?tag=aedes-aegypti. Acesso em: 27 mar. 2026.

PARANÁ. Secretaria de Estado da Saúde (SESA). Informe Epidemiológico — Dengue. Disponível em: https://www.dengue.pr.gov.br/Pagina/Boletins-da-Dengue. Acesso em: 27 mar. 2026.

TAUIL, Pedro Luis. Aspectos críticos do controle do dengue no Brasil. Cadernos de Saúde Pública, Rio de Janeiro, v. 18, n. 3, p. 867-871, maio/jun. 2002.

ZIVIANI, Nivio. Projeto de Algoritmos com implementações em Java e C++. São Paulo: Cengage Learning, 2011.